

复旦大学计算机科学技术学院

《C++程序设计》期末考试试卷

B 卷 共 7 页

课程代码: COMP130135

考试形式: ☐ 开卷 ☒ 闭卷

2017 年 7 月

(本试卷答卷时间为 120 分钟, 答案必须写在答题纸上, 做在试卷上无效)

专业 _____ 学号 _____ 姓名 _____

题号	一	二	三	四	总分
分值	24	20	24	32	100

一、选择题 (每题只选择一个答案; 如选多个, 不计分。每题 3 分, 总分 24 分)

- (1) 关于 list 和 vector, 下列说法中错误的是_____。
- A) list 顺序访问比 vector 稍慢些
 - B) vector 不支持在中间某位置插入元素的操作。
 - C) 对于大型数据, list 删除和插入要比 vector 快得多
 - D) vector 支持位置索引, 而 list 不支持
- (2) 关于构造函数, 下列说法错误的是_____。
- A) 构造函数名字和类名相同
 - B) 构造函数在创建对象时自动执行
 - C) 构造函数无任何函数返回类型
 - D) 构造函数有且只有一个
- (3) C++派生类可以访问其基类的_____。
- A) 公有成员
 - B) 保护成员
 - C) 私有成员
 - D) 公有和保护成员
- (4) 下列关于运算符重载的描述中, 错误的是_____。
- A) 运算符重载不可以改变操作数的个数。
 - B) 运算符重载不可以改变运算符的功能。
 - C) 运算符重载不可以改变结合方向。
 - D) 运算符重载不可以改变运算优先级。
- (5) 下列关于类的描述中, 错误的是_____。
- A) 一般来说, 类可以控制对象在创建、复制、赋值和销毁时的所有行为。
 - B) 如果一个类没有定义构造函数, 编译系统就会合成默认构造函数。
 - C) 在构造函数中分配资源的类, 几乎都必须定义复制构造函数、赋值操作符和析构函数。
 - D) 如果这个类没有明确的定义复制构造函数、赋值操作符以及析构函数, 编译器不会自动

合成这些操作。

(6) 定义析构函数时，应该注意_____。

- A) 无形参，也不可重载
- B) 返回类型是 void 类型
- C) 其名与类名完全相同
- D) 函数体中必须有 delete 语句

(7) 下列关于虚函数和动态绑定的说法，错误的是_____。

- A) 动态绑定：虚函数在运行时，通过引用或者指针来调用这个函数。
- B) 静态绑定：通过一个对象来调用一个虚函数，可以在编译时知道这个对象的准确类型。
- C) C++是通过虚函数的动态绑定特性来支持多态的。
- D) 派生类不能再定义新的虚函数。

(8) 执行下面的程序将输出_____。

```
#include <iostream.h>
class BASE{
    char b;
public:
    BASE(char n):b(n){}
    virtual ~BASE(){cout<<b;}
};
class DERIVED:public BASE{
    char d;
public:
    DERIVED(char n):BASE(n+1),d(n){}
    ~DERIVED(){cout<<d;}
};
int main(){
    DERIVED a('X');
    return 0;
}
```

- A) X
- B) Y
- C) XY
- D) YX

二、程序阅读题（每题 5 分，共 20 分）

1、根据以下 C++ 代码写出 main 函数运行后的输出结果

<pre>#include <iostream> using std::cout; class A{ int a; public: A():a(0){static n=0; cout << ++n; }; A(const A & cA){static n=0; cout << ++n; }; operator=(const A & cA){static n=0; cout << ++n; }; };</pre>	<pre>int main() { A b, c; A d=b, e=c; b=e; return 0; }</pre>
---	---

2、根据以下 C++ 代码写出 main 函数运行后的输出结果

<pre>#include <iostream> using std::cout; using std::ostream; const int Max_Num=100; template<class T> class X{ int Num; T Data[Max_Num]; public: X(int n=0){ Num=n; }; void Add(const T & d){ Data[Num++]=d; }; T & Del(){ return Data[Num--]; }; friend ostream & operator<<(ostream& os, const X<T> & x); };</pre>	<pre>template<class T> ostream & operator<<(ostream & os, const X<T> & x){ for(int i=0;i<x.Num;i++) os << x.Data[i]; return os; } int main() { X<int> x; for(int i=0;i<10;i++) x.Add(i); for(i=0;i<5;i++) x.Del(); cout << x ; return 0; }</pre>
---	---

3、根据以下 C++ 代码写出 main 函数运行后的输出结果

<pre>#include <iostream> using std::cout; using std::ostream; const int Max_Num=100; class X{ int Num; char Vec[Max_Num];</pre>	<pre>X operator+(const X & cX1, const X & cX2){ int i, carry=0; X x; for(i=0;i<cX1.Num i<cX2.Num;i++){ int v1=i<cX1.Num ? cX1.Vec[i]-'0' : 0; int v2=i<cX2.Num ? cX2.Vec[i]-'0' : 0; x.Vec[i]=(v1+v2+carry)%10+'0';</pre>
---	--

<pre> public: X(int n=1, const char * v="0"){ Num=n; for(int i=0;i<Num;i++) Vec[i]=v[i]; }; friend X operator+(const X & cX1, const X & cX2); friend ostream & operator<<(ostream& os, const X & x); }; ostream & operator<<(ostream & os, const X & x){ for(int i=x.Num-1;i>=0;i--) os << x.Vec[i]; return os; } </pre>	<pre> carry=(v1+v2+carry)/10; } while(carry){ x.Vec[i]=(x.Vec[i]+carry)%10; carry=(x.Vec[i]+carry)/10; i++; } x.Num=i; return x; } int main() { X x(3,"789").y(9, "123456789"); cout << x+y ; return 0; } </pre>
---	--

4. 阅读以下 C++程序:

<pre> #include <iostream> using std::cout; using std::endl; class Animal{ public: virtual char * getColor()const{ return "None"; }; int getWeight()const{ return 0; }; }; class Horse: public Animal{ public: char * getColor()const{ return "Brown"; }; virtual int getWeight()const{ return 500; }; }; </pre>	<pre> class Sheep: public Animal{ public: char * getColor()const{ return "White"; }; int getWeight()const{ return 300; }; }; int main() { Sheep s; Horse h; Animal a1,&a2=s,&a3=h; cout <<a1.getColor()<<"-"<<a1.getWeight()<<endl; cout <<a2.getColor()<<"-"<<a2.getWeight()<<endl; cout <<a3.getColor()<<"-"<<a3.getWeight()<<endl; return 0; } </pre>
--	--

三、程序填空题（每空 3 分，共 24 分）

1. 在空格中填上代码，使得类A完整，并且程序运行将输出：“53103”。

```

#include <iostream>
using std::cout;
using std::ostream;
class A{
    int a;
public:
    A(int b=0){

```

```

        a=b;
    };
    (1.1) operator+=(const A& cA){
        a+=cA.a;
        return *this;
    };
    (1.2) operator+(const A& cA,const A& cB){
        return A(cA.a+cB.a);
    };
    friend ostream & operator<<(ostream & os, const A & cA){
        os << cA.a;
        (1.3);
    };
};

```

```

int main( )
{
    A b(2),c(3),d;
    d+=b+=c;
    d= (1.4) +c;
    cout << b << c << d;
    return 0;
}

```

2.在空格中填上代码,使得类Window和类House完整,并且程序运行将输出:“Rectangle”。

```

#include <iostream>
using std::cout;
class Window{
    char * style;
public:
    Window(char * s="None"){style=s;};
    Window & operator=(const Window & w){
        if( (2.1) ){
            style=w.style;
        }
        return *this;
    };
    const char * getStyle() const{
        return style;
    };
};
class House{
    Window w;
public:

```

```

House(const Window &w){
    _____(2.2)_____;
};
const char * getWindowStyle()const{
    return _____(2.3)_____;
};
};

int main( )
{
    House h(_____(2.4)_____);
    cout << h.getWindowStyle();
    return 0;
}

```

四、编程题（共 32 分）

1.（12 分）

编写模板函数实现冒泡排序。main 函数如下，程序将输出“0123456789ehors”

```

int main( )
{
    int a[]={9,8,7,6,5,4,3,2,1,0},i;
    sort(10,a);
    for(i=0;i<10;i++)
        cout << a[i];
    char c[]="horse";
    sort(5, c);
    for(i=0; i<5; i++)
        cout << c[i];
    return 0;
}

```

2.（20 分）

按要求用 C++实现表示桌子的类 Table，表示桌腿的类 Leg 和桌面的类 Desktop。main 函数及其输出如下所示

```

int main( )
{
    Leg legs[]={Leg(70,"Red"),Leg(70,"Yellow"),Leg(70,"Blue"),Leg(70,"Green")};
    Desktop top(60,80,"White");
    Table table(top,legs);
    cout << table;
    return 0;
}
//输出:

```

This table has a White 80*60 desktop.

Its legs are as follow:

Red 70

Yellow 70

Blue 70

Green 70

- 1) Desktop 类中 double 类型的 width 和 Length 分别表示桌子的宽和长，const char * 类型的 color 表示颜色。

(a) 实现 1 个公有构造函数：

构造函数： `Desktop(double l=0,double w=0,const char *c="")`

(b) 此外，Desktop 类的成员函数还包括：

返回长度的公有函数： `getLength`

返回宽度的公有函数： `getWidth`

返回颜色的公有函数： `getColor`

- 2) Leg 类中 double 类型的 Height 表示桌腿的高度，const char * 类型的 color 表示颜色。

(a) 实现 1 个公有构造函数：

构造函数： `Leg(double h=0, const char * c="")`

(b) 此外，Leg 类的成员函数还包括：

返回高度的公有函数： `getHeight`

返回颜色的公有函数： `getColor`

- 3) 类 Table 组合了类 Desktop 和类 Leg，一张桌子 (Table) 有一个桌面和四个桌腿 (Leg)。

(a) 实现 1 个公有构造函数：

构造函数： `Table(const Desktop & t, const Leg ls[])`

(b) 此外，还实现 Table 类的友函数：

显示信息的输出函数： `friend ostream & operator<<(ostream &os, const Table & t)`